

Zakipoint Health Internship – Day 3

Name: Dipendra Raj Bhatt

1. Python Programming

1.0.1 Python Fundamentals

- Variables
- Loops
- Functions
- Methods

1.2 Logical Thinking & Coding Problems

- Practice solving algorithmic problems
- Break down problems into smaller steps

1.3 Object-Oriented Programming (OOP)

- Classes
- Inheritance
- Polymorphism
- Magic Methods

1.4 Decorators and Generators

- Learn function decorators

- Use generators for memory-efficient loops

1.5 Exception Handling & Logging

- Try/Except blocks
- Logging best practices

1.6 File Handling

- Reading & writing files
- Working with CSV, JSON

1.7 Writing Clean, Modular Code

- Functions and modules
- Code readability and PEP8

1.4 Decorators and Generators

Learn function decorators |
Use generators for memory-efficient loops

Decorators

A **decorator** is a function that takes another function as an argument and returns a modified function.

1. Used to **extend function behavior without modifying the original function**.
2. Applied using the **@decorator_name** syntax.
3. Uses a **wrapper function** inside the decorator.
4. Often uses ***args** and ****kwargs** to support any parameters.
5. Common in **logging, authentication, caching, and timing functions**.

Syntax

```
def decorator_name(function_name):  
    def wrapper(*args, **kwargs):  
        # additional functionality  
        result = function_name(*args, **kwargs)  
        # additional functionality  
        return result  
    return wrapper
```

```
@decorator_name  
def function_name(parameters):  
    pass
```

Generators

A **generator** is a function that returns an iterator and produces values **one at a time** using the `yield` keyword.

1. Uses **yield** instead of `return`.
2. Gives values **one by one**, only when needed.
3. The function **stops at yield** and continues from there next time.
4. Uses **less memory** because it generates values one by one.
5. You can get values using a **for loop** or the **next()** function.

Syntax

```
def generator_name(parameters):  
    # initialization  
    while condition:  
        yield value
```

```
In [1]: def decorator(function):  
        def wrapper():  
            print("Transaction Initiated")  
            function()  
            print("Transaction Completed")
```

```
    return wrapper

def hello():
    print("Executing all steps of transaction")

dec = decorator(hello)
dec()
```

Transaction Initiated
Executing all steps of transaction
Transaction Completed

In [2]: # Generators

```
def creator():
    i=1
    while i<= 200:

        yield i
        i+=1

print(creator())

X= creator()
print(next(X))
print(next(X))

print(list(X))
```

<generator object creator at 0x7da4444831c0>

1

2

```
[3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33
, 34, 35, 36, 37, 38, 39, 40, 41, 42, 43, 44, 45, 46, 47, 48, 49, 50, 51, 52, 53, 54, 55, 56, 57, 58, 59, 60, 61, 62
, 63, 64, 65, 66, 67, 68, 69, 70, 71, 72, 73, 74, 75, 76, 77, 78, 79, 80, 81, 82, 83, 84, 85, 86, 87, 88, 89, 90, 91
, 92, 93, 94, 95, 96, 97, 98, 99, 100, 101, 102, 103, 104, 105, 106, 107, 108, 109, 110, 111, 112, 113, 114, 115, 11
6, 117, 118, 119, 120, 121, 122, 123, 124, 125, 126, 127, 128, 129, 130, 131, 132, 133, 134, 135, 136, 137, 138, 139
, 140, 141, 142, 143, 144, 145, 146, 147, 148, 149, 150, 151, 152, 153, 154, 155, 156, 157, 158, 159, 160, 161, 162,
163, 164, 165, 166, 167, 168, 169, 170, 171, 172, 173, 174, 175, 176, 177, 178, 179, 180, 181, 182, 183, 184, 185, 1
86, 187, 188, 189, 190, 191, 192, 193, 194, 195, 196, 197, 198, 199, 200]
```

Exception Handling in Python

Introduction

Exception Handling is a mechanism in Python that allows a program to handle runtime errors without crashing. When an error occurs during execution, Python generates an **exception**. If the exception is not handled, the program stops and displays an error message.

Using exception handling helps developers write **robust, reliable, and user-friendly programs**.

An **exception** is an event that occurs during the execution of a program that interrupts the normal flow of instructions.

It usually happens due to situations such as invalid user input, dividing by zero, accessing a file that does not exist, or using incorrect data types.

Why Exception Handling is Important

- It prevents programs from crashing unexpectedly.
- It allows programs to continue running even when errors occur.
- It helps developers identify and debug issues in code.
- It provides meaningful error messages to users.
- It improves the reliability and stability of applications.
- It helps handle unexpected inputs or situations gracefully.

Syntax

```
try:
    # code that may cause an exception
except ExceptionType:
    # code to handle the exception
```

Exception handling in Python is performed using **try** and **except** blocks.

The **try block** contains the code that might cause an error.

The **except block** contains the code that handles the error if it occurs.

Block	Purpose
try	Contains code that might cause an exception
except	Handles the exception if it occurs

syntax

```
try:
    # code that may cause an exception
except ExceptionType:
    # code to handle the exception
---
```

The execution flow of exception handling works as follows:

1. The program first enters the **try block**.
2. If no error occurs, the program skips the **except block**.
3. If an error occurs, Python immediately jumps to the corresponding **except block**.
4. The exception is handled and the program continues execution.

Handling Multiple Exceptions

syntax

```
try:
    # risky code
except ExceptionType1:
    # handle exception type 1
except ExceptionType2:
    # handle exception type 2
```

Using the Else Block

The **else block** is executed only when the code inside the **try block** runs successfully without raising any exceptions.

This block is useful when some code should run only if no errors occur in the try block.

syntax

```
try: # risky code except ExceptionType: # handle exception else: # executes if no exception occurs
```

Using the Finally Block

The **finally block** always executes whether an exception occurs or not.

It is commonly used for **cleanup operations**, such as:

- Closing files
- Releasing system resources
- Closing database connections

The finally block ensures that important cleanup code always runs.

syntax

```
try:
    # risky code
except ExceptionType:
    # handle exception
finally:
    # always executes
```

Complete Exception Handling Structure

- **try block** – contains risky code

- **except block** – handles exceptions
- **else block** – executes if no exception occurs
- **finally block** – always executes

Using Exception Objects

Sometimes developers need to know the exact error message generated by Python.

Exception objects store detailed information about the error that occurred. By capturing this object, the program can display or log the specific error message. **syntax**

```
try:  
    # risky code  
except Exception as e:  
    # access the exception message
```

Raising Exceptions

Python allows developers to manually generate exceptions using the **raise** keyword.

Raising an exception is useful when a program detects an invalid condition and wants to stop execution or notify the user about the problem.

This technique helps enforce rules and validate data in applications.

syntax

```
raise ExceptionType("Error message")
```

Common Built-in Exceptions

Exception	Description
ZeroDivisionError	Occurs when dividing by zero

Exception	Description
ValueError	Occurs when an invalid value is provided
TypeError	Occurs when an incorrect data type is used
IndexError	Occurs when accessing a list index that does not exist
KeyError	Occurs when accessing a dictionary key that does not exist
FileNotFoundError	Occurs when attempting to open a file that does not exist

14. Use Cases of Exception Handling

Exception handling is widely used in real-world programming applications.

1. User Input Validation Used to ensure users provide valid data and prevent invalid input from crashing the program.

```
try:
    age = int(input("Enter your age: "))
    print("Age:", age)

except ValueError:
    print("Please enter a valid number")
```

2. File Handling Used when opening or reading files to handle situations where the file may not exist.

```
try:
    file = open("data.txt", "r")
    print(file.read())

except FileNotFoundError:
    print("File does not exist")
```

3. Network or API Requests Used to handle network failures, timeouts, or connection issues when accessing external services.

```
try:
    data = fetch_data()
except ConnectionError:
    print("Network connection failed")
```

4. Database Operations Used to handle database connection failures or query errors.

```
try:
    data = fetch_data()
except ConnectionError:
    print("Network connection failed")
```

In [7]: *# Exception handling*

```
try:
    num = int(input("Enter a number:"))
    result = 10/num
    print(result)

except ZeroDivisionError:
    print("Cannot be divided by zero")
```

2.5

In [8]:

```
try:
    number = int(input("Enter number: "))
    print("You entered:", number)

except ValueError:
    print("Invalid input. Please enter a number.")
```

You entered: 0

In [9]:

```
try:
    num = int(input("Enter number: "))
    result = 10 / num
    print(result)

except ZeroDivisionError:
    print("Division by zero is not allowed")

except ValueError:
    print("Invalid input entered")
```

1.6666666666666667

In [11]:

```
try:
    num = int(input("Enter number: "))
```

```
    result = 10 / num

except ZeroDivisionError:
    print("Division by zero error")

else:
    print("Result is:", result)
```

Division by zero error

```
In [12]: try:
        file = open("data.txt", "r")
        content = file.read()
        print(content)

        except FileNotFoundError:
            print("File not found")

        finally:
            print("Execution completed")
```

```
1.0000000000000000e+01
2.0000000000000000e+01
3.0000000000000000e+01
```

Execution completed

```
In [13]: try:
    num = int(input("Enter number: "))
    result = 10 / num

    except ZeroDivisionError:
        print("Cannot divide by zero")

    except ValueError:
        print("Invalid input")

    else:
        print("Result:", result)

    finally:
        print("Program finished")
```

Cannot divide by zero
Program finished

```
In [14]: try:
    num = int(input("Enter number: "))
    result = 10 / num

    except Exception as e:
        print("An error occurred:", e)
```

```
In [15]: age = int(input("Enter your age: "))

    if age < 18:
        raise ValueError("Age must be 18 or above")

    print("Access granted")
```

Access granted

Python Logging

Logging is used to record events that happen while a program runs.

It helps developers debug issues, track program behavior, and monitor applications.

In Python, logging is handled using the `logging` module.

print() vs logging

print()	logging
Simple output	Structured output
No severity levels	Multiple log levels
Hard to manage	Easy to disable/enable
Not used in production	Industry standard

Syntax

General syntax for logging:

```
logging.<level>(message)
```

Example levels:

- `logging.debug()`
- `logging.info()`
- `logging.warning()`
- `logging.error()`
- `logging.critical()`

Logging Levels

Level	Purpose
DEBUG	Detailed debugging information
INFO	General program events
WARNING	Something unexpected happened

Level	Purpose
ERROR	A function failed or error occurred
CRITICAL	Serious failure in the program

Logger Object Syntax

Instead of using the root logger, create a logger.

Syntax:

```
logger = logging.getLogger(name)
```

Common usage:

```
logger = logging.getLogger(__name__)
```

Logging with logger:

```
logger.info(message)
```

```
logger.error(message)
```

Basic Logging Configuration

Logging configuration controls how logs are displayed.

Syntax:

```
logging.basicConfig(level=LEVEL)
```

Example configuration options:

- level
- filename

- format
-

Log Message Format Syntax

Format string syntax:

```
"%(asctime)s - %(name)s - %(levelname)s - %(message)s"
```

Common formatting fields:

Field	Meaning
<code>%(asctime)s</code>	Time of log
<code>%(name)s</code>	Logger name
<code>%(levelname)s</code>	Log level
<code>%(message)s</code>	Log message

Logging to a File

Logs can be stored in files instead of console.

Syntax option:

```
filename="app.log"
```

Purpose:

- persistent log records
 - debugging production systems
 - monitoring applications
-

Exception Logging

Logging exceptions helps capture error details.

Syntax:

```
logger.exception(message)
```

Purpose:

- records error message
 - includes stack trace
-

Log Handlers

Handlers define where logs go.

Common handler types:

- Console handler
- File handler
- Rotating file handler

Syntax example concept:

```
logger.addHandler(handler)
```

Log Rotation

Sometimes log files become very large when an application runs for a long time. To prevent this problem, Python provides `RotatingFileHandler`, which automatically creates new log files when the current log file reaches a size limit.

Concept:

- limit log file size
- create backup log files

Important parameters:

- maxBytes
 - backupCount
-

Best Practices for Python Logging

- Use logging instead of print statements
 - Use appropriate logging levels
 - Create module-specific loggers
 - Configure logging once at application start
 - Write clear log messages
 - Avoid logging sensitive data
 - Use log rotation for large applications
 - Log exceptions properly
-

Logging Workflow

Typical steps in Python logging:

1. Import logging module
 2. Configure logging
 3. Create logger
 4. Log messages using appropriate level
-

```
In [16]: import logging
# basic logging message
logging.debug("Debug message")
logging.info("Information message")
logging.warning("Warning message")
logging.error("Error message")
logging.critical("Critical message")
```

```
WARNING:root:Warning message
ERROR:root:Error message
CRITICAL:root:Critical message
```

```
In [17]: # configuring logging levels
logging.basicConfig(level=logging.DEBUG)
logging.debug("Debug Message")
logging.info("Info Message")
logging.warning("Warning message")
```

```
WARNING:root:Warning message
```

```
In [18]: # creating Logger object, best for real applications
logger = logging.getLogger(__name__)
logger.info("Logger Created")
logging.warning("Warning message")
```

```
WARNING:root:Warning message
```

```
In [19]: # custom log message format
logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s - %(name)s - %(levelname)s - %(message)s"
)

logger = logging.getLogger(__name__)

logger.info("Application started")
```

```
In [20]: # Logging to file
logging.basicConfig(
    filename="app.log",
    level=logging.INFO,
    format="%(asctime)s - %(levelname)s - %(message)s"
)

logging.info("Program started")
logging.warning("Something might be wrong")
```

```
WARNING:root:Something might be wrong
```

```
In [21]: #File Handler
logging.basicConfig(level=logging.INFO)

logger = logging.getLogger(__name__)

try:
    x = 10 / 0
except Exception:
    logger.exception("An error occurred")
```

```
ERROR:__main__:An error occurred
Traceback (most recent call last):
  File "/tmp/ipykernel_36137/1504785415.py", line 7, in <module>
    x = 10 / 0
        ~~~^~~
ZeroDivisionError: division by zero
```

```
In [22]: #console handler

logger = logging.getLogger("console_logger")
logger.setLevel(logging.INFO)

console_handler = logging.StreamHandler()

logger.addHandler(console_handler)

logger.info("This will appear in console")
```

```
This will appear in console
INFO:console_logger:This will appear in console
```

```
In [23]: # Rotating Log files
import logging
from logging.handlers import RotatingFileHandler

logger = logging.getLogger("rotating_logger")
logger.setLevel(logging.INFO)

handler = RotatingFileHandler(
    "app.log",
    maxBytes=2000,
```

```
        backupCount=3
    )

    logger.addHandler(handler)

    logger.info("This message goes to rotating log file")
```

INFO:rotating_logger:This message goes to rotating log file

```
In [24]: import logging

logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s - %(levelname)s - %(message)s"
)

logger = logging.getLogger("app")

logger.info("Application started")

user = "admin"

logger.info("User %s logged in", user)

try:
    result = 10 / 2
    logger.debug("Calculation successful")
except Exception:
    logger.exception("Calculation failed")

logger.info("Application finished")
```

1.6 File Handling

File handling in Python allows programs to **read, write, and manage files** stored on the system. It is commonly used for storing program data, reading configuration files, and saving results.

Opening a File

Syntax: `file = open("filename", "mode")`

File Modes

Mode	Description
"r"	Read file (default mode)
"w"	Write to file (creates or overwrites the file)
"a"	Append data to the file
"x"	Create a new file

Reading a File

```
file = open("data.txt", "r")
content = file.read()
print(content)
file.close()
```

Writing to a File

```
file = open("data.txt", "w")
file.write("Hello Python")
file.close()
```

Using with

The `with` statement automatically closes the file after use.

```
with open("data.txt", "r") as file:
    content = file.read()
    print(content)
```

Exception Handling with Files

```
try:
    with open("data.txt", "r") as file:
        print(file.read())
except FileNotFoundError:
    print("File not found")
```

```
In [25]: file = open("data.txt", "w")
file.write("Hello Python")
file.close()
```

```
In [26]: try:
    with open("data.txt", "r") as file:
        print(file.read())
except FileNotFoundError:
    print("File not found")
```

Hello Python

```
In [27]: # Creating and writing to a file
with open("data.txt", "w") as file:
    file.write("Hello Python\n")
    file.write("File handling example\n")

print("File created and data written")

# Appending new data to the file
with open("data.txt", "a") as file:
    file.write("Appending a new line\n")

print("Data appended to file")

# Reading the entire file
with open("data.txt", "r") as file:
    content = file.read()
    print("\nFile Content:")
    print(content)

# Reading file line by line
print("Reading file line by line:")
with open("data.txt", "r") as file:
    for line in file:
        print(line.strip())

# Exception handling while opening file
try:
    with open("missing.txt", "r") as file:
```

```
        print(file.read())
    except FileNotFoundError:
        print("Error: File does not exist")
```

File created and data written
Data appended to file

File Content:
Hello Python
File handling example
Appending a new line

Reading file line by line:
Hello Python
File handling example
Appending a new line
Error: File does not exist

```
In [28]: # CSV files
# Reading CSV files row by row

import csv

with open("Demo.csv - Sheet1.csv", "r") as file:
    reader = csv.reader(file)

    for row in reader:
        print(row)
```

```
['ID', 'Name']
['1', 'Ram']
['2', 'Fam']
```

```
In [30]: # Using Dictionaries
with open("Demo.csv - Sheet1.csv", "r") as file:
    reader = csv.DictReader(file)

    for row in reader:
        print(row["ID"], row["Name"])
```

```
1 Ram
2 Fam
```

PEP8(Python Style Guide)

PEP8 is the official style guide for writing Python code. It provides conventions for formatting Python programs so that code is consistent and readable.

Use 4 spaces for indentation

Limit lines to 79 characters

Use snake_case for variable and function names

Use UPPER_CASE for constants

Add spaces around operators

Leave blank lines between functions and classes

```
In [31]: # Constant written in UPPER_CASE
TAX_RATE = 0.15

def calculate_total(price, quantity):
    total_price = price * quantity
    tax_amount = total_price * TAX_RATE
    final_price = total_price + tax_amount
    return final_price

def display_total(price, quantity):
    result = calculate_total(price, quantity)
    print("Total price:", result)

item_price = 100
item_quantity = 2

display_total(item_price, item_quantity)
```


Total price: 230.0

Python Data Structures

In **Python**, data structures are used to **store, organize, and manage data efficiently**.

The four main built-in Python data structures are:

1. List
 2. Tuple
 3. Set
 4. Dictionary
-

1. List

A **List** is an ordered, mutable (changeable) collection of items. Lists allow duplicate values and can store different data types.

Characteristics

- Ordered
- Mutable (can be modified)
- Allows duplicate values
- Supports multiple data types

syntax

```
list_name = [item1, item2, item3]
```

Common List Methods

Method	Meaning
<code>append()</code>	Adds an element to the end of the list
<code>insert()</code>	Inserts an element at a specific position
<code>remove()</code>	Removes the first occurrence of an item

Method	Meaning
<code>pop()</code>	Removes and returns item by index
<code>sort()</code>	Sorts the list in ascending order
<code>reverse()</code>	Reverses the order of list elements

2. Tuple

A **Tuple** is an ordered, immutable (unchangeable) collection of items. Tuples allow duplicate values and can store different data types.

Characteristics

- Ordered
- Immutable (cannot be modified)
- Allows duplicate values
- Faster than lists

syntax

```
tuple_name = (item1, item2, item3)
```

Common Tuple Methods

Method	Meaning
<code>count()</code>	Counts how many times an element appears
<code>index()</code>	Returns the index of a specified element

3. Set

A **Set** is an unordered collection of unique elements. Duplicate values are automatically removed.

Characteristics

- Unordered

- Mutable
- Does not allow duplicate values
- Fast membership testing

syntax

```
set_name = {item1, item2, item3}
```

Common Set Methods

Method	Meaning
<code>add()</code>	Adds an element to the set
<code>remove()</code>	Removes a specific element
<code>union()</code>	Combines two sets
<code>intersection()</code>	Returns common elements between sets
<code>difference()</code>	Returns elements present in one set but not the other

4. Dictionary

A **Dictionary** stores data in key–value pairs. Each key is unique and is used to access its corresponding value.

Characteristics

- Mutable
- Keys must be unique
- Fast lookup using keys
- Ordered (Python 3.7+)

syntax

```
dict_name = {  
    key1: value1,  
    key2: value2
```

```
}
```

Common Dictionary Methods

Method	Meaning
<code>get()</code>	Returns the value of a specified key
<code>keys()</code>	Returns all keys in the dictionary
<code>values()</code>	Returns all values in the dictionary
<code>items()</code>	Returns key–value pairs
<code>pop()</code>	Removes a key and its value

```
In [32]: ### List Example

# Creating a list
fruits = ["apple", "banana", "mango"]

# append()
fruits.append("orange")

# insert()
fruits.insert(1, "grapes")

# remove()
fruits.remove("banana")

# pop()
fruits.pop(2)

# sort()
numbers = [5, 2, 8, 1]
numbers.sort()

# reverse()
numbers.reverse()

print(fruits)
```

```
print(numbers)

### Tuple Example

# Creating a tuple
numbers = (1, 2, 2, 3, 4)

# count()
print(numbers.count(2))

# index()
print(numbers.index(3))

### Set Example

# Creating sets
A = {1, 2, 3}
B = {3, 4, 5}

# add()
A.add(6)

# remove()
A.remove(2)

# union()
print(A.union(B))

# intersection()
print(A.intersection(B))

# difference()
print(A.difference(B))

### Dictionary Example

# Creating a dictionary
student = {
    "name": "Ram",
```

```
        "age": 20
    }

    # get()
    print(student.get("name"))

    # keys()
    print(student.keys())

    # values()
    print(student.values())

    # items()
    print(student.items())

    # pop()
    student.pop("age")

    print(student)
```

['apple', 'grapes', 'orange']

[8, 5, 2, 1]

2

3

{1, 3, 4, 5, 6}

{3}

{1, 6}

Ram

dict_keys(['name', 'age'])

dict_values(['Ram', 20])

dict_items([('name', 'Ram'), ('age', 20)])

{'name': 'Ram'}

Data Foundation

2.1 NumPy

2.2 Pandas

2.3 Data collection (CSV, Excel, APIs, databases, web scraping)

2.3 Data cleaning & wrangling

2.4 Exploratory Data Analysis (EDA)

2.5 Matplotlib & Seaborn

Feature	Python Array	NumPy Array	Pandas (Series/DataFrame)
Homogeneous	Yes	Yes	Series: Yes, DF: Mixed
Mutable	Yes	Yes	Yes
Size	Fixed	Fixed (after creation)	Flexible (rows/cols can be added)
Multi-dimensional	Limited (1D)	Yes	Yes (DataFrame 2D)
Vectorized Operations	No	Yes	Yes (via NumPy)
Labels / Index	No	No	Yes
Use case	Simple numbers	Scientific computing	Data analysis, tabular data

NumPy

NumPy (Numerical Python) is a Python library for **numerical computing**.

It provides:

- Efficient **N-dimensional arrays** (`ndarray`)
- Fast **mathematical operations** on arrays
- Tools for **linear algebra, statistics, and random number generation**

Why use NumPy?

It allows **vectorized operations**, meaning you can operate on whole arrays **without explicit loops**, making computations **faster and more memory-efficient**.

Key Features

- **N-dimensional arrays (ndarray)** – Generalization of vectors and matrices
 - **Vectorized operations** – Apply operations element-wise without loops
 - **Broadcasting** – Automatic alignment of differently shaped arrays
 - **Mathematical functions** – Arithmetic, linear algebra, statistics
 - **Random numbers** – For simulations and ML
 - **Integration with C/C++/Fortran** – High-performance computations
 - **Foundation for scientific computing** – Used in **pandas, SciPy, TensorFlow, PyTorch**, etc.
-

```
In [33]: pip install numpy
```

Requirement already satisfied: numpy in ./lib/python3.12/site-packages (2.4.3)
Note: you may need to restart the kernel to use updated packages.

```
In [34]: # Creating Numpy Arrays
import numpy as np
array = np.array([1,3,3,4,5,])
print(array)
```

```
[1 3 3 4 5]
```

```
In [35]: array1= np.array([1,3, 3.2, 3])
array2 = np.array([1,3,3, 3.2])
final_array= array1 + array2
final_array
```

```
Out[35]: array([2. , 6. , 6.2, 6.2])
```

```
In [36]: array3= np.array([(1,3, 3.2, 6],
                          [1,3, 3.2, 3]))

array3
```

```
Out[36]: array([[1. , 3. , 3.2, 6. ],
               [1. , 3. , 3.2, 3. ]])
```

```
In [37]: array4= np.array([(1,3, 3.2, 6],
                          [1,3, 3.2, 31],
```



```
[1,3, 3.2, 3.2],  
[1,3, 3.2, 3]))
```

```
array4
```

```
Out[37]: array([[ 1. ,  3. ,  3.2,  6. ],  
               [ 1. ,  3. ,  3.2, 31. ],  
               [ 1. ,  3. ,  3.2,  3.2],  
               [ 1. ,  3. ,  3.2,  3. ]])
```

```
In [38]: array4.shape
```

```
Out[38]: (4, 4)
```

```
In [39]: array4.ndim
```

```
Out[39]: 2
```

```
In [40]: array4.dtype
```

```
Out[40]: dtype('float64')
```

```
In [41]: np.zeros(4)
```

```
Out[41]: array([0., 0., 0., 0.])
```

```
In [42]: np.zeros((2,5))
```

```
Out[42]: array([[0., 0., 0., 0., 0.],  
               [0., 0., 0., 0., 0.]])
```

```
In [43]: s=(2,2)  
         np.zeros(s)
```

```
Out[43]: array([[0., 0.],  
               [0., 0.]])
```

```
In [44]: np.linspace(10,100, 10)
```

```
Out[44]: array([ 10.,  20.,  30.,  40.,  50.,  60.,  70.,  80.,  90., 100.] )
```

```
In [45]: np.eye(2,2)# returns diaonal 1
```

```
Out[45]: array([[1., 0.],  
               [0., 1.]])
```

```
In [46]: # Array Operations  
# Arithmetic operations (+, -, *, /)  
# Scalar operations  
  
arr = np.array([1, 2, 3, 4])  
  
print(arr + 10)    # Add 10 to each element  
print("\n_____\n")  
print(arr - 2)     # Subtract 2 from each element  
print("\n_____\n")  
print(arr * 3)     # Multiply each element by 3  
print("\n_____\n")  
print(arr / 2)     # Divide each element by 2  
print("\n_____\n")  
# Comparison operations  
a = np.array([1, 2, 3])  
b = np.array([2, 2, 2])  
print(a > b)  
print("\n_____\n")  
print(a < b)  
print("\n_____\n")  
print(a == b)  
print("\n_____\n")  
print(a != b)  
print("\n_____\n")
```

```
[11 12 13 14]
```

```
[-1  0  1  2]
```

```
[ 3  6  9 12]
```

```
[0.5 1.  1.5 2. ]
```

```
[False False  True]
```

```
[ True False False]
```

```
[False  True False]
```

```
[ True False  True]
```

```
In [47]: # Indexing and Slicing
# Slice arrays arr[start:stop:step]
arr= np.array([12,24,26,48,65,90,2,3,5,90])
print(arr[1:4])
print(arr[0:8:4])

# Boolean indexing
print(arr[arr>20])
```

```
# Fancy indexing  
print(arr[[0,3,5,7]])
```

```
[24 26 48]  
[12 65]  
[24 26 48 65 90 90]  
[12 48 90  3]
```

```
In [48]: # ArrayIndexing  
arr = np.array([[1, 2, 3],  
                [4, 5, 6],  
                [4, 6, 2],  
                [9, 4, 5]  
              ])  
  
print(arr[0, 1])  
print(arr[:, 1])  
  
print(arr[3:,])
```

```
2  
[2 5 6 4]  
[[9 4 5]]
```

```
In [49]: # 5. Reshaping Arrays  
# reshape()  
arr.reshape(2,6)
```

```
Out[49]: array([[1, 2, 3, 4, 5, 6],  
               [4, 6, 2, 9, 4, 5]])
```

Function	Type	Description
flatten()	Copy	Creates a new array
ravel()	View	Refers to original array

```
In [50]: # flatten()  
arr.flatten()
```

```
Out[50]: array([1, 2, 3, 4, 5, 6, 4, 6, 2, 9, 4, 5])
```

```
In [51]: arr = np.array([[1, 2, 3], [4, 5, 6]])
```

```
   .ravel_arr = arr.ravel()
    print(ravel_arr)
```

```
[1 2 3 4 5 6]
```

```
In [52]: arr = np.array([1, 2, 3, 4, 5, 6])
```

```
    print(arr.reshape(3, -1)) # here -1 calculates col dimension automatically
```

```
[[1 2]
 [3 4]
 [5 6]]
```

```
In [ ]:
```

```
In [53]: # Linear algebra
# 1.Matrix multiplication
A = np.array([[1, 2],
              [3, 4]])
B = np.array([[5, 6],
              [5, 8]])
print(np.dot(A,B))
print("\n_____ \n")
print(np.matmul(A,B))
print("\n_____ \n")
# 2.Transpose of matrix
print(A.T)
print("\n_____ \n")
# 3.Determinant of matrix
print(np.linalg.det(A))
print("\n_____ \n")
# 4.Inverse of matrix
print(np.linalg.inv(A))
print("\n_____ \n")
# 5.Eigenvalues and Eigenvectors
values, vectors = np.linalg.eig(A)
print("Values=",values)
```

```
print("Vectors=", vectors)
print("\n_____ \n")
# 6.Solving Linear Equation
print(np.linalg.solve(A,B))

print("\n_____ \n")
# 7.Normalization of a vector
v = np.array([3, 4])

print(np.linalg.norm(v))
```

```
[[15 22]
 [35 50]]
```

```
[[15 22]
 [35 50]]
```

```
[[1 3]
 [2 4]]
```

```
-2.0000000000000004
```

```
[[-2.  1. ]
 [ 1.5 -0.5]]
```

```
Values= [-0.37228132  5.37228132]
Vectors= [[-0.82456484 -0.41597356]
 [ 0.56576746 -0.90937671]]
```

```
[[-5. -4.]
 [ 5.  5.]]
```

```
5.0
```

```
In [54]: # Polinomial in numpy
p = np.polyld([2,3,1]) #2x2 + 3x + 1 → [2, 3, 1]
print(p)
```

```
print(p(2))    # substitute x = 2
print(p.roots)
```

```
      2
2 x + 3 x + 1
15
[-1.  -0.5]
```

In [55]: *#Statistics*

```
# Sample data
arr = np.array([10, 20, 30, 40, 50])

# Basic statistics
print("Array:", arr)
print("Mean:", np.mean(arr))
print("Median:", np.median(arr))
print("Standard Deviation:", np.std(arr))
print("Variance:", np.var(arr))
print("Minimum:", np.min(arr))
print("Maximum:", np.max(arr))
print("Sum:", np.sum(arr))

# 2D array for axis operations
arr2d = np.array([[1, 2, 3],
                  [4, 5, 6]])

print("\n2D Array:\n", arr2d)

# Axis operations
print("Sum (column-wise):", np.sum(arr2d, axis=0))
print("Sum (row-wise):", np.sum(arr2d, axis=1))

print("Mean (column-wise):", np.mean(arr2d, axis=0))
print("Mean (row-wise):", np.mean(arr2d, axis=1))
```



```
Array: [10 20 30 40 50]
Mean: 30.0
Median: 30.0
Standard Deviation: 14.142135623730951
Variance: 200.0
Minimum: 10
Maximum: 50
Sum: 150
```

```
2D Array:
[[1 2 3]
 [4 5 6]]
Sum (column-wise): [5 7 9]
Sum (row-wise): [ 6 15]
Mean (column-wise): [2.5 3.5 4.5]
Mean (row-wise): [2. 5.]
```

```
In [56]: # File Handling with numpy
# Saving Array to file(.npz)
arr = np.array([1,2,3,4])
np.save("data.npy",arr)
```

```
In [57]: # Loading Array from file
loaded_arr= np.load("data.npy")
print(loaded_arr)

[1 2 3 4]
```

```
In [58]: # Saving multiple arrays(.npz)
a = np.array([1, 2, 3])
b = np.array([4, 5, 6])

np.savez("data.npz", arr1=a, arr2=b)
```

```
In [59]: # Loading multiple arrays(.npz)
data = np.load("data.npz")

print(data["arr1"])
print(data["arr2"])
```

```
[1 2 3]
[4 5 6]
```

```
In [60]: # Saving to Text File(.txt)
arr = np.array([10, 20, 30])
np.savetxt("data.txt", arr)
```

```
In [61]: # Loading from text file
loaded = np.loadtxt("data.txt")

print(loaded)
```

```
[10. 20. 30.]
```

Pandas

Pandas is a powerful Python library for data manipulation and analysis.

It provides:

- Flexible data structures for handling structured data
- Tools for data cleaning, transformation, and analysis
- Fast and efficient operations on labeled data

Why use Pandas?

It allows you to work with structured data (like tables or spreadsheets) in an intuitive way, enabling fast data analysis, cleaning, and transformation with minimal code. It is widely used in data science, machine learning, and data analytics.

Key Features

- **Series and DataFrame** – Core data structures for 1D and 2D labeled data
- **Labeled data** – Each row and column has labels, making data easier to understand and manipulate
- **Data alignment** – Automatic alignment of data based on labels
- **Handling missing data** – Built-in tools to detect, remove, or fill missing values (**NaN**)
- **Data manipulation** – Filtering, grouping, aggregation, merging, and reshaping data

- **GroupBy operations** – Split, apply, and combine data efficiently
- **Time series support** – Powerful tools for working with dates and time-based data
- **File I/O** – Read/write data from CSV, Excel, SQL, JSON, etc.
- **Indexing and selection** – Powerful ways to select and slice data (`loc` , `iloc`)
- **Vectorized operations** – Fast computations without explicit loops
- **Integration** – Works seamlessly with NumPy, Matplotlib, SciPy, and ML libraries
- **Performance** – Built on top of NumPy for fast computation

In [62]: `pip install pandas`

```
Requirement already satisfied: pandas in ./lib/python3.12/site-packages (3.0.1)
Requirement already satisfied: numpy>=1.26.0 in ./lib/python3.12/site-packages (from pandas) (2.4.3)
Requirement already satisfied: python-dateutil>=2.8.2 in ./lib/python3.12/site-packages (from pandas) (2.9.0.post0)
Requirement already satisfied: six>=1.5 in ./lib/python3.12/site-packages (from python-dateutil>=2.8.2->pandas) (1.17.0)
Note: you may need to restart the kernel to use updated packages.
```

In [63]: `import pandas as pd`
Pandas Basic DataStructure
1.Series: 1D labeled array holding data of any type
`s = pd.Series([1,3,5,np.nan,6,8])`
`s`

Out[63]:

0	1.0
1	3.0
2	5.0
3	NaN
4	6.0
5	8.0

dtype: float64

In [64]: *# 2.DataFrame: 2D data structure*
`df = pd.DataFrame(`
 `{`
 `"A": 1.0,`
 `"B": pd.Timestamp("20260102"),`
 `"C": pd.Series(1, index=list(range(4)), dtype="float32"),`
 `"D": np.array([3] * 4, dtype="int32"),`

```
        "E": pd.Categorical(["test", "train", "test", "train"]),
        "F": "foo",
    }
)
df
```

```
Out[64]:
```

	A	B	C	D	E	F
0	1.0	2026-01-02	1.0	3	test	foo
1	1.0	2026-01-02	1.0	3	train	foo
2	1.0	2026-01-02	1.0	3	test	foo
3	1.0	2026-01-02	1.0	3	train	foo

```
In [65]: df.head()
```

```
Out[65]:
```

	A	B	C	D	E	F
0	1.0	2026-01-02	1.0	3	test	foo
1	1.0	2026-01-02	1.0	3	train	foo
2	1.0	2026-01-02	1.0	3	test	foo
3	1.0	2026-01-02	1.0	3	train	foo

```
In [66]: df.tail(2)
```

```
Out[66]:
```

	A	B	C	D	E	F
2	1.0	2026-01-02	1.0	3	test	foo
3	1.0	2026-01-02	1.0	3	train	foo

```
In [67]: df.index
```

```
Out[67]: Index([0, 1, 2, 3], dtype='int64')
```

```
In [68]: df.to_numpy()
```

```
Out[68]: array([[1.0, Timestamp('2026-01-02 00:00:00'), 1.0, 3, 'test', 'foo'],
                [1.0, Timestamp('2026-01-02 00:00:00'), 1.0, 3, 'train', 'foo'],
                [1.0, Timestamp('2026-01-02 00:00:00'), 1.0, 3, 'test', 'foo'],
                [1.0, Timestamp('2026-01-02 00:00:00'), 1.0, 3, 'train', 'foo']],
          dtype=object)
```

```
In [69]: df.describe()
```

```
Out[69]:
```

	A	B	C	D
count	4.0	4	4.0	4.0
mean	1.0	2026-01-02 00:00:00	1.0	3.0
min	1.0	2026-01-02 00:00:00	1.0	3.0
25%	1.0	2026-01-02 00:00:00	1.0	3.0
50%	1.0	2026-01-02 00:00:00	1.0	3.0
75%	1.0	2026-01-02 00:00:00	1.0	3.0
max	1.0	2026-01-02 00:00:00	1.0	3.0
std	0.0	NaN	0.0	0.0

```
In [70]: df.T
```

```
Out[70]:
```

	0	1	2	3
A	1.0	1.0	1.0	1.0
B	2026-01-02 00:00:00	2026-01-02 00:00:00	2026-01-02 00:00:00	2026-01-02 00:00:00
C	1.0	1.0	1.0	1.0
D	3	3	3	3
E	test	train	test	train
F	foo	foo	foo	foo

```
In [71]: df.sort_index(axis=1, ascending=False)
```

```
Out[71]:
```

	F	E	D	C	B	A
0	foo	test	3	1.0	2026-01-02	1.0
1	foo	train	3	1.0	2026-01-02	1.0
2	foo	test	3	1.0	2026-01-02	1.0
3	foo	train	3	1.0	2026-01-02	1.0

```
In [72]: df.loc[0]
```

```
Out[72]:
```

A	1.0
B	2026-01-02 00:00:00
C	1.0
D	3
E	test
F	foo

Name: 0, dtype: object

```
In [73]: df.iloc[0]
```

```
Out[73]:
```

A	1.0
B	2026-01-02 00:00:00
C	1.0
D	3
E	test
F	foo

Name: 0, dtype: object

```
In [74]: df.sort_index(axis=1,)
```

```
Out[74]:
```

	A	B	C	D	E	F
0	1.0	2026-01-02	1.0	3	test	foo
1	1.0	2026-01-02	1.0	3	train	foo
2	1.0	2026-01-02	1.0	3	test	foo
3	1.0	2026-01-02	1.0	3	train	foo

```
In [75]: df= pd.read_csv("bmw_global_sales_2018_2025.csv")
df
```

```
Out[75]:
```

	Year	Month	Region	Model	Units_Sold	Avg_Price_EUR	Revenue_EUR	BEV_Share	Premium_Share	GDP_Growth	Fuel_Price
0	2018	1	Europe	3 Series	7822	47482	371404204	0.011	19.12	3.50	
1	2018	1	Europe	5 Series	10280	61685	634121800	0.019	19.12	3.50	
2	2018	1	Europe	X3	3105	58433	181434465	0.022	19.12	3.50	
3	2018	1	Europe	X5	7420	67955	504226100	0.021	19.12	3.50	
4	2018	1	Europe	X7	8474	92300	782150200	0.035	19.12	3.50	
...	...	...	...	...	...	...	...	...	...	...	...
3067	2025	12	RestOfWorld	X5	9281	68198	632945638	0.201	5.89	2.37	
3068	2025	12	RestOfWorld	X7	12785	91839	1174161615	0.203	5.89	2.37	
3069	2025	12	RestOfWorld	i4	4906	63437	311221922	0.180	5.89	2.37	
3070	2025	12	RestOfWorld	iX	7871	73867	581407157	0.196	5.89	2.37	
3071	2025	12	RestOfWorld	MINI	6732	41811	281471652	0.167	5.89	2.37	

3072 rows × 11 columns

```
In [76]: new_df = df[["Year", "Month", "Region", "Model", "Units_Sold", "Avg_Price_EUR" ]]
```

```
new_df
```

Out[76]:

	Year	Month	Region	Model	Units_Sold	Avg_Price_EUR
0	2018	1	Europe	3 Series	7822	47482
1	2018	1	Europe	5 Series	10280	61685
2	2018	1	Europe	X3	3105	58433
3	2018	1	Europe	X5	7420	67955
4	2018	1	Europe	X7	8474	92300
...	...	...	...	...	...	...
3067	2025	12	RestOfWorld	X5	9281	68198
3068	2025	12	RestOfWorld	X7	12785	91839
3069	2025	12	RestOfWorld	i4	4906	63437
3070	2025	12	RestOfWorld	iX	7871	73867
3071	2025	12	RestOfWorld	MINI	6732	41811

3072 rows × 6 columns